

Projects ×

Files

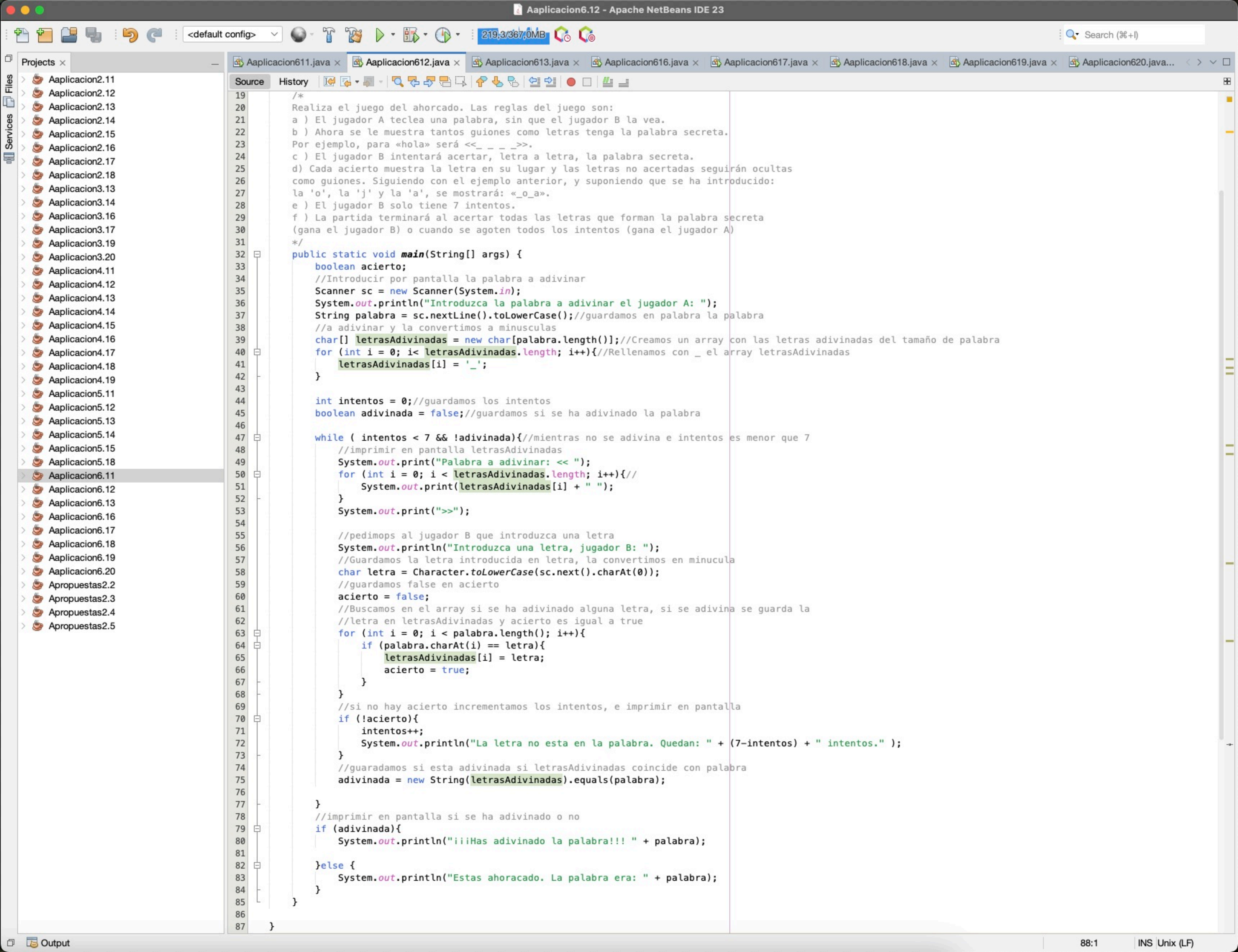
Services

- >  Aplicacion2.11
- >  Aplicacion2.12
- >  Aplicacion2.13
- >  Aplicacion2.14
- >  Aplicacion2.15
- >  Aplicacion2.16
- >  Aplicacion2.17
- >  Aplicacion2.18
- >  Aplicacion3.13
- >  Aplicacion3.14
- >  Aplicacion3.16
- >  Aplicacion3.17
- >  Aplicacion3.19
- >  Aplicacion3.20
- >  Aplicacion4.11
- >  Aplicacion4.12
- >  Aplicacion4.13
- >  Aplicacion4.14
- >  Aplicacion4.15
- >  Aplicacion4.16
- >  Aplicacion4.17
- >  Aplicacion4.18
- >  Aplicacion4.19
- >  Aplicacion5.11
- >  Aplicacion5.12
- >  Aplicacion5.13
- >  Aplicacion5.14
- >  Aplicacion5.15
- >  Aplicacion5.18
- >  Aplicacion6.11
- >  Aplicacion6.12
- >  Aplicacion6.13
- >  Aplicacion6.16
- >  Aplicacion6.17
- >  Aplicacion6.18
- >  Aplicacion6.19
- > Aplicacion6.20
- > Apropuestas2.2
- > Apropuestas2.3
- > Apropuestas2.4
- > Apropuestas2.5

```
Aplicacion611.java x Aplicacion612.java x Aplicacion613.java x Aplicacion616.java x Aplicacion617.java x Aplicacion618.java x Aplicacion619.java x Aplicacion620.java...
Source History
1 ...4 lines
5 package aplicacion6.pkg11;
6
7 import java.util.Scanner;
8
9 /**...4 lines */
13 public class Aplicacion611 {
14
15     /**...3 lines */
18     /*
19      Escribe un programa descodificador que muestre un texto codificado con el programa
20      realizado en la Actividad resuelta 6.11.
21      Intercambiar las constantes
22      */
23     public static void main(String[] args) {
24         Scanner sc = new Scanner (System.in) ;
25         final char conjunto1[] = {'e', 'i', 'k', 'm', 'p', 'q', 'r', 's', 't', 'u', 'v'};
26         final char conjunto2[] = {'p', 'v', 'i', 'u', 'm', 't', 'e', 'r', 'k', 'q', 's'};
27         char codificado[]; //tabla que contendrá la codificación del texto introducido
28         String texto;
29         System.out.print("Introduzca un texto a decodificar: ");
30         texto = sc.nextLine();
31         texto = texto.toLowerCase(); //convertimos el texto a minúscula, para poder
32         //codificar las mayúsculas y las minúsculas con el mismo conjunto.
33         codificado = new char[texto.length()]; //creamos una tabla de igual tamaño
34         for (int i = 0; i < texto.length(); i++) { // recorremos el texto a codificar
35             //codificamos el i-ésimo carácter del texto
36             codificado[i] = codifica(conjunto1, conjunto2, texto.charAt(i));
37             //modificamos el programa haciendo la llamada a la función codifica
38             //intercambiando conjunto1 y conjunto2 para que decodifique
39         }
40         texto = String.valueOf(codificado); //convertimos la tabla con la codificación
41         //en una cadena
42         System.out.println(texto);
43     }
44
45     static char codifica(char conjunto1[], char conjunto2[], char c){
46         final String conj1 = String.valueOf(conjunto1); //conj1 es un String con los
47         //elementos de la tabla conjunto1. Facilita la búsqueda
48         char codificado; //carácter codificado correspondiente a c
49         int pos = conj1.indexOf(c); //buscamos c en el conjunto 1. Al ser conj1 una
50         //cadena, indexOf() busca por nosotros. En otro caso, tendríamos que buscar
51         //mediante un bucle un elemento en la tabla
52         if (pos == -1) { //si no hemos encontrado c en conj1
53             codificado = c; //no podemos codificar, devolvemos c
54         } else{
55             codificado = conjunto2[pos]; //pos marca la posición de c en conjunto1
56             //entonces elegimos el correspondiente conjunto2
57         }
58         return codificado;
59     }
60 }
61
```

Output - Aplicacion6.11 (run) ×

```
run:
Introduzca un texto a decodificar: matqvko
paquito
BUILD SUCCESSFUL (total time: 8 seconds)
```



run:

Introduzca la palabra a adivinar el jugador A:

aplicaciones

Palabra a adivinar: << _ _ _ _ _ >>Introduzca una letra, jugador B:

a

Palabra a adivinar: << a _ _ _ a _ _ _ _ >>Introduzca una letra, jugador B:

e

Palabra a adivinar: << a _ _ _ a _ _ _ e _ >>Introduzca una letra, jugador B:

i

Palabra a adivinar: << a _ _ i _ a _ i _ _ e _ >>Introduzca una letra, jugador B:

o

Palabra a adivinar: << a _ _ i _ a _ i o _ e _ >>Introduzca una letra, jugador B:

l

Palabra a adivinar: << a _ l i _ a _ i o _ e _ >>Introduzca una letra, jugador B:

f

La letra no esta en la palabra. Quedan: 6 intentos.

Palabra a adivinar: << a _ l i _ a _ i o _ e _ >>Introduzca una letra, jugador B:

p

Palabra a adivinar: << a p l i _ a _ i o _ e _ >>Introduzca una letra, jugador B:

c

Palabra a adivinar: << a p l i c a c i o _ e _ >>Introduzca una letra, jugador B:

u

La letra no esta en la palabra. Quedan: 5 intentos.

Palabra a adivinar: << a p l i c a c i o _ e _ >>Introduzca una letra, jugador B:

n

Palabra a adivinar: << a p l i c a c i o n e _ >>Introduzca una letra, jugador B:

s

iiiHas adivinado la palabra!!! aplicaciones

BUILD SUCCESSFUL (total time: 1 minute 15 seconds)

328,2/367,0MB

Search (%+l)

Projects

Aplicacion2.11

Aplicacion2.12

Aplicacion2.13

Aplicacion2.14

Aplicacion2.15

Aplicacion2.16

Aplicacion2.17

Aplicacion2.18

Aplicacion3.13

Aplicacion3.14

Aplicacion3.16

Aplicacion3.17

Aplicacion3.19

Aplicacion3.20

Aplicacion4.11

Aplicacion4.12

Aplicacion4.13

Aplicacion4.14

Aplicacion4.15

Aplicacion4.16

Aplicacion4.17

Aplicacion4.18

Aplicacion4.19

Aplicacion5.11

Aplicacion5.12

Aplicacion5.13

Aplicacion5.14

Aplicacion5.15

Aplicacion5.18

Aplicacion6.11

Aplicacion6.12

Aplicacion6.13

Aplicacion6.16

Aplicacion6.17

Aplicacion6.18

Aplicacion6.19

Aplicacion6.20

Apropuestas2.2

Apropuestas2.3

Apropuestas2.4

Apropuestas2.5

Aplicacion611.java

Aplicacion612.java

Aplicacion613.java

Aplicacion616.java

Aplicacion617.java

Aplicacion618.java

Aplicacion619.java

Aplicacion620.java...

Source

History

...

1

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

...4 lines

package aplicacion6.pkg13;

/**

*

* @author daldo

*/

public class Aplicacion613 {

/**

* @param args the command line arguments

*/

/**

El preprocesador del lenguaje C elimina los comentarios (/* .*//*) del código fuente antes

de compilar. Diseña un programa que lea por teclado una sentencia en C, y elimine los

comentarios.

Sentencia: if (a==3) /* igual a tres */ a++; /* incrementamos a */

Salida: if (a==3) a++;

*/

public static void main(String[] args) {

//texto para quitar comentarios

String cadena1 = "if (a==3) /* igual a tres */ a++; /* incrementamos a */";

int i1 = cadena1.indexOf("/*"); //guardamos en i1 la posición de el inicio del comentario

while(i1 != -1){ //mientras i1 sea distinto de -1

int i2 = cadena1.indexOf("*/"); //guardamos en i2 el indice del fin de comentario

//a cadena 1 le quitamos la parte del comentario

cadena1 = cadena1.substring(0, i1-1) + cadena1.substring(i2+2);

//guardamos en i1 el inicio de otro comentario

i1 = cadena1.indexOf("/*");

}

//imprimir cadena1

System.out.println(cadena1);

}

}

Output - Aplicacion6.13 (run)

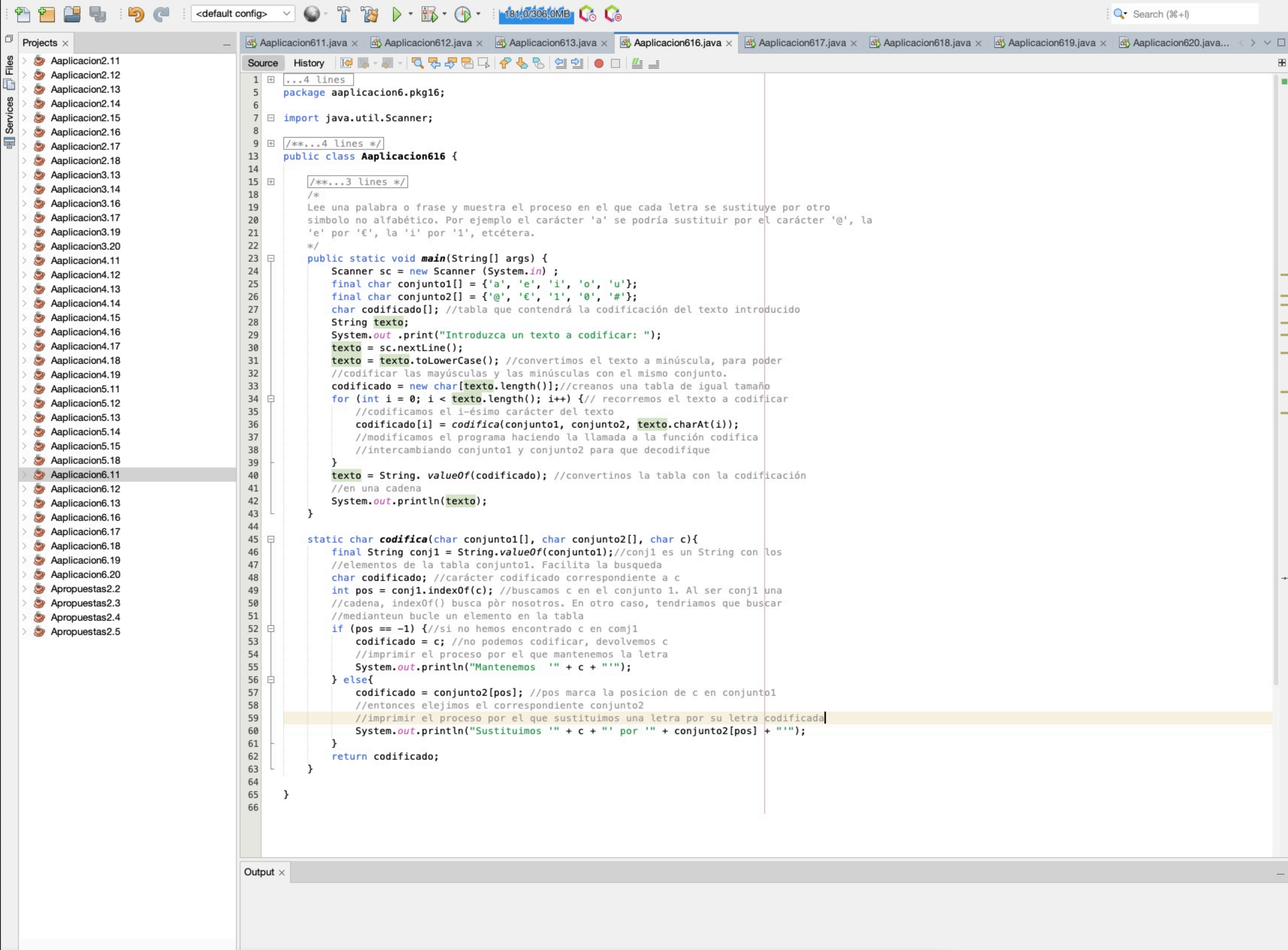
run:

if (a==3) a++:

BUILD SUCCESSFUL (total time: 0 seconds)

34:27

INS Unix (LF)



Projects x

- > Aplicacion2.11
- > Aplicacion2.12
- > Aplicacion2.13
- > Aplicacion2.14
- > Aplicacion2.15
- > Aplicacion2.16
- > Aplicacion2.17
- > Aplicacion2.18
- > Aplicacion3.13
- > Aplicacion3.14
- > Aplicacion3.16
- > Aplicacion3.17
- > Aplicacion3.19
- > Aplicacion3.20
- > Aplicacion4.11
- > Aplicacion4.12
- > Aplicacion4.13
- > Aplicacion4.14
- > Aplicacion4.15
- > Aplicacion4.16
- > Aplicacion4.17
- > Aplicacion4.18
- > Aplicacion4.19
- > Aplicacion5.11
- > Aplicacion5.12
- > Aplicacion5.13
- > Aplicacion5.14
- > Aplicacion5.15
- > Aplicacion5.18
- > Aplicacion6.11
- > Aplicacion6.12
- > Aplicacion6.13
- > Aplicacion6.16
- > Aplicacion6.17
- > Aplicacion6.18
- > Aplicacion6.19
- > Aplicacion6.20
- > Aplicacion6.21
- > Aplicacion6.22
- > Aplicacion6.23
- > Aplicacion6.24
- > Aplicacion6.25
- > Aplicacion6.26
- > Aplicacion6.27
- > Aplicacion6.28
- > Aplicacion6.29
- > Aplicacion6.30
- > Aplicacion6.31
- > Aplicacion6.32
- > Aplicacion6.33
- > Aplicacion6.34
- > Aplicacion6.35
- > Aplicacion6.36
- > Aplicacion6.37
- > Aplicacion6.38
- > Aplicacion6.39
- > Aplicacion6.40
- > Aplicacion6.41
- > Aplicacion6.42
- > Aplicacion6.43
- > Aplicacion6.44
- > Aplicacion6.45
- > Aplicacion6.46
- > Aplicacion6.47
- > Aplicacion6.48
- > Aplicacion6.49
- > Aplicacion6.50
- > Aplicacion6.51
- > Aplicacion6.52
- > Aplicacion6.53
- > Aplicacion6.54
- > Aplicacion6.55
- > Aplicacion6.56
- > Aplicacion6.57
- > Aplicacion6.58
- > Aplicacion6.59
- > Aplicacion6.60
- > Aplicacion6.61
- > Aplicacion6.62
- > Aplicacion6.63
- > Aplicacion6.64
- > Aplicacion6.65
- > Aplicacion6.66

Source History

```
1 ...4 lines
5 package aplicacion6.pkg16;
6
7 import java.util.Scanner;
8
9 /**...4 lines */
13 public class Aplicacion616 {
14
15     /**...3 lines */
16     /*
17     Lee una palabra o frase y muestra el proceso en el que cada letra se sustituye por otro
18     simbolo no alfabético. Por ejemplo el carácter 'a' se podría sustituir por el carácter '@', la
19     'e' por '€', la 'i' por '1', etcétera.
20     */
21     public static void main(String[] args) {
22         Scanner sc = new Scanner (System.in) ;
23         final char conjunto1[] = {'a', 'e', 'i', 'o', 'u'};
24         final char conjunto2[] = {'@', '€', '1', '0', '#'};
25         char codificado[]; //tabla que contendrá la codificación del texto introducido
26         String texto;
27         System.out .print("Introduzca un texto a codificar: ");
28         texto = sc.nextLine();
29         texto = texto.toLowerCase(); //convertimos el texto a minúscula, para poder
30         //codificar las mayúsculas y las minúsculas con el mismo conjunto.
31         codificado = new char[texto.length()]; //creamos una tabla de igual tamaño
32         for (int i = 0; i < texto.length(); i++) { // recorremos el texto a codificar
33             //codificamos el i-ésimo carácter del texto
34             codificado[i] = codifica(conjunto1, conjunto2, texto.charAt(i));
35             //modificamos el programa haciendo la llamada a la función codifica
36             //intercambiando conjunto1 y conjunto2 para que decodifique
37         }
38         texto = String. valueOf(codificado); //convertinos la tabla con la codificación
39         //en una cadena
40         System.out.println(texto);
41     }
42
43     static char codifica(char conjunto1[], char conjunto2[], char c){
44         final String conj1 = String.valueOf(conjunto1); //conj1 es un String con los
45         //elementos de la tabla conjunto1. Facilita la busqueda
46         char codificado; //carácter codificado correspondiente a c
47         int pos = conj1.indexOf(c); //buscamos c en el conjunto 1. Al ser conj1 una
48         //cadena, indexOf() busca por nosotros. En otro caso, tendríamos que buscar
49         //medianteun bucle un elemento en la tabla
50         if (pos == -1) { //si no hemos encontrado c en conj1
51             codificado = c; //no podemos codificar, devolvemos c
52             //imprimir el proceso por el que mantenemos la letra
53             System.out.println("Mantenemos " + c + "");
54         } else{
55             codificado = conjunto2[pos]; //pos marca la posicion de c en conjunto1
56             //entonces elejimos el correspondiente conjunto2
57             //imprimir el proceso por el que sustituimos una letra por su letra codificada
58             System.out.println("Sustituimos " + c + " por " + conjunto2[pos] + "");
59         }
60         return codificado;
61     }
62 }
63
64
65
66
```

Output x

59:91 INS Unix (LF)


```
run:
Introduzca un texto a codificar: Lee una palabra o frase y muestra el proceso
Mantenemos 'l'
Sustituimos 'e' por '€'
Sustituimos 'e' por '€'
Mantenemos ' '
Sustituimos 'u' por '#'
Mantenemos 'n'
Sustituimos 'a' por '@'
Mantenemos ' '
Mantenemos 'p'
Sustituimos 'a' por '@'
Mantenemos 'l'
Sustituimos 'a' por '@'
Mantenemos 'b'
Mantenemos 'r'
Sustituimos 'a' por '@'
Mantenemos ' '
Sustituimos 'o' por '0'
Mantenemos ' '
Mantenemos 'f'
Mantenemos 'r'
Sustituimos 'a' por '@'
Mantenemos 's'
Sustituimos 'e' por '€'
Mantenemos ' '
Mantenemos 'y'
Mantenemos ' '
Mantenemos 'm'
Sustituimos 'u' por '#'
Sustituimos 'e' por '€'
Mantenemos 's'
Mantenemos 't'
Mantenemos 'r'
Sustituimos 'a' por '@'
Mantenemos ' '
Sustituimos 'e' por '€'
Mantenemos 'l'
Mantenemos ' '
Mantenemos 'p'
Mantenemos 'r'
Sustituimos 'o' por '0'
Mantenemos 'c'
Sustituimos 'e' por '€'
Mantenemos 's'
Sustituimos 'o' por '0'
l€€ #n@ p@l@br@ 0 fr@s€ y m#€str@ €l pr0c€s0
BUILD SUCCESSFUL (total time: 3 seconds)
```

Projects ×

- > Aplicacion2.11
- > Aplicacion2.12
- > Aplicacion2.13
- > Aplicacion2.14
- > Aplicacion2.15
- > Aplicacion2.16
- > Aplicacion2.17
- > Aplicacion2.18
- > Aplicacion3.13
- > Aplicacion3.14
- > Aplicacion3.16
- > Aplicacion3.17
- > Aplicacion3.19
- > Aplicacion3.20
- > Aplicacion4.11
- > Aplicacion4.12
- > Aplicacion4.13
- > Aplicacion4.14
- > Aplicacion4.15
- > Aplicacion4.16
- > Aplicacion4.17
- > Aplicacion4.18
- > Aplicacion4.19
- > Aplicacion5.11
- > Aplicacion5.12
- > Aplicacion5.13
- > Aplicacion5.14
- > Aplicacion5.15
- > Aplicacion5.18
- > Aplicacion6.11
- > Aplicacion6.12
- > Aplicacion6.13
- > Aplicacion6.16
- > Aplicacion6.17
- > Aplicacion6.18
- > Aplicacion6.19
- > Aplicacion6.20
- > Apropuestas2.2
- > Apropuestas2.3
- > Apropuestas2.4
- > Apropuestas2.5

```
1  ...4 lines
5  package aplicacion6.pkg17;
6
7  import java.util.Scanner;
8
9  /**
10   *
11   * @author daldo
12   */
13  public class Aplicacion617 {
14
15      /**...3 lines */
16      /*
17       Construir un programa que convierta una palabra en secuencias de n letras. Por ejemplo,
18       la palabra «destornillador», dividida en secuencias de 4 letras, se mostrará de la siguien-
19       te forma:
20       dest
21       orni
22       llad
23       or
24       */
25      public static void main(String[] args) {
26          //Introducir la palabra y el tamaño de las secuencias
27          Scanner sc = new Scanner(System.in);
28          System.out.println("Introduzca una palabra");
29          String palabra = sc.nextLine();
30
31          System.out.println("Introduzca el tamaño n de las secuencias en la que se quiere dividir la palabra");
32          int n = sc.nextInt();
33          //llamamos a el procedimiento divPalabra pasandole la palabra y el tamaño de las secuencias
34          divPalabra(palabra, n);
35
36      }
37
38      static void divPalabra(String palabra, int n){
39          int l = palabra.length();//guardamos en l la cantidad de caracteres de la palabra
40          for (int i = 0; i < l; i += n){//recorremos la palabra hasta l-1
41              if (i + n <= l){//si el indice + el largo de la secuencia es menor o igual que l
42                  //hacemos un substring desde i hasta i+n
43                  System.out.println(palabra.substring(i, i+ n));
44              } else{//hacemos un substring desde i
45                  System.out.println(palabra.substring(i));
46              }
47          }
48      }
49  }
50
51  }
52
53  }
```

```
Output - Aplicacion6.17 (run) x
run:
Introduzca una palabra
destornillador
Introduzca el tamaño n de las secuncias en la que se quiere dividir la palabra
4
dest
orni
llad
or
BUILD SUCCESSFUL (total time: 11 seconds)
```


Projects x

- > Aaplicacion2.11
- > Aaplicacion2.12
- > Aaplicacion2.13
- > Aaplicacion2.14
- > Aaplicacion2.15
- > Aaplicacion2.16
- > Aaplicacion2.17
- > Aaplicacion2.18
- > Aaplicacion3.13
- > Aaplicacion3.14
- > Aaplicacion3.16
- > Aaplicacion3.17
- > Aaplicacion3.19
- > Aaplicacion3.20
- > Aaplicacion4.11
- > Aaplicacion4.12
- > Aaplicacion4.13
- > Aaplicacion4.14
- > Aaplicacion4.15
- > Aaplicacion4.16
- > Aaplicacion4.17
- > Aaplicacion4.18
- > Aaplicacion4.19
- > Aaplicacion5.11
- > Aaplicacion5.12
- > Aaplicacion5.13
- > Aaplicacion5.14
- > Aaplicacion5.15
- > Aaplicacion5.18
- > Aaplicacion6.11
- > Aaplicacion6.12
- > Aaplicacion6.13
- > Aaplicacion6.16
- > Aaplicacion6.17
- > Aaplicacion6.18
- > Aaplicacion6.19
- > Aaplicacion6.20
- > Apropuestas2.2
- > Apropuestas2.3
- > Apropuestas2.4
- > Apropuestas2.5

Aaplicacion611.java x Aaplicacion612.java x Aaplicacion613.java x Aaplicacion616.java x Aaplicacion617.java x Aaplicacion618.java x Aaplicacion619.java x Aaplicacion620.java...

Source History

```
1  ...4 lines
5  package aplicacion6.pkg18;
6
7  import java.util.Arrays;
8  import java.util.Scanner;
9
10 /**...4 lines */
14 public class Aaplicacion618 {
15
16     /**
17      * @param args the command line arguments
18      */
19     /**
20      * Escribe una aplicación que convierte una frase (que puede estar escrita con cualquier
21      * combinación de mayúsculas y minúsculas) en el nombre de una variable que utilice la no-
22      * menclatura Camel. Por ejemplo, la frase «Me Gusta merenDAR gaLLEtas», se convertirá
23      * en «meGustaMerendar Galletas».
24      */
25     public static void main(String[] args) {
26         Scanner sc = new Scanner(System.in);
27         System.out.println("Introduzca una frase: ");
28         String frase = sc.nextLine();
29         //guardamos en frase convirtiendo la primera letra a minúscula
30         frase = Character.toLowerCase(frase.charAt(0)) + frase.substring(1);
31         String palabras[] = frase.split(" "); //pasamos la frase a un array separando
32         //las palabras por el espacio en blanco
33         for (int i = 1; i < palabras.length; i++) { //Recorremos cada palabra menos la primera y
34             //ponemos la primera letra en mayúsculas
35             palabras[i] = Character.toUpperCase(palabras[i].charAt(0)) + palabras[i].substring(1);
36         }
37         //imprimir la nueva palabra
38         for (int i = 0; i < palabras.length; i++) {
39             System.out.print(palabras[i]);
40         }
41         System.out.println();
42     }
43 }
44
45 }
46
```

Output - Aaplicacion6.18 (run) x

```
run:
Introduzca una frase:
Palabra adivinada por letras
palabraAdivinadaPorLetras
BUILD SUCCESSFUL (total time: 23 seconds)
```


Search (%+l)

Projects ×

- > Aaplicacion2.11
- > Aaplicacion2.12
- > Aaplicacion2.13
- > Aaplicacion2.14
- > Aaplicacion2.15
- > Aaplicacion2.16
- > Aaplicacion2.17
- > Aaplicacion2.18
- > Aaplicacion3.13
- > Aaplicacion3.14
- > Aaplicacion3.16
- > Aaplicacion3.17
- > Aaplicacion3.19
- > Aaplicacion3.20
- > Aaplicacion4.11
- > Aaplicacion4.12
- > Aaplicacion4.13
- > Aaplicacion4.14
- > Aaplicacion4.15
- > Aaplicacion4.16
- > Aaplicacion4.17
- > Aaplicacion4.18
- > Aaplicacion4.19
- > Aaplicacion5.11
- > Aaplicacion5.12
- > Aaplicacion5.13
- > Aaplicacion5.14
- > Aaplicacion5.15
- > Aaplicacion5.18
- > Aaplicacion6.11
- > Aaplicacion6.12
- > Aaplicacion6.13
- > Aaplicacion6.16
- > Aaplicacion6.17
- > Aaplicacion6.18
- > Aaplicacion6.19
- > Aaplicacion6.20
- > Apropuestas2.2
- > Apropuestas2.3
- > Apropuestas2.4
- > Apropuestas2.5

Source History

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template
4  */
5  package aaplicacion6.pkg19;
6
7  import java.util.Scanner;
8
9  /**
10   *
11   * @author daldo
12   */
13  public class Aaplicacion619 {
14
15      /**
16       * @param args the command line arguments
17       */
18      /*
19       Implementa un sencillo editor de texto que, una vez que se ha introducido el texto,
20       permita reemplazar todas las ocurrencias de una palabra por otra.
21       */
22      public static void main(String[] args) {
23          //pedimos que se introduzca el texto
24          Scanner sc = new Scanner(System.in);
25          System.out.println("Introduzca un texto: ");
26          String texto = sc.nextLine();
27          //pedimos que se intraduzca la palabra a buscar
28          System.out.println("Introduzca la palabra a buscar: ");
29          String palabra = sc.nextLine();
30          //pedimos que se introduzca la palabra que va a reemplazar
31          System.out.println("Introduzca la palabra con la que quiere reemplazar");
32          String palabraNueva = sc.nextLine();
33          //dentro del texto reemplazamos todas las ocurrencias de palabra por palabraNueva
34          texto = texto.replaceAll(palabra, palabraNueva);
35          //imprimir el nuevo texto
36          System.out.println(texto);
37      }
38  }
39
40  }
41
```

Output - Aaplicacion6.19 (run) ×

```
run:
Introduzca un texto:
Hola esto es un ejemplo de texto sencillo
Introduzca la palabra a buscar:
texto
Introduzca la palabra con la que quiere reemplazar
frase
Hola esto es un ejemplo de frase sencillo
BUILD SUCCESSFUL (total time: 1 minute 26 seconds)
```


- Projects
- > Aplicacion2.11
 - > Aplicacion2.12
 - > Aplicacion2.13
 - > Aplicacion2.14
 - > Aplicacion2.15
 - > Aplicacion2.16
 - > Aplicacion2.17
 - > Aplicacion2.18
 - > Aplicacion3.13
 - > Aplicacion3.14
 - > Aplicacion3.16
 - > Aplicacion3.17
 - > Aplicacion3.19
 - > Aplicacion3.20
 - > Aplicacion4.11
 - > Aplicacion4.12
 - > Aplicacion4.13
 - > Aplicacion4.14
 - > Aplicacion4.15
 - > Aplicacion4.16
 - > Aplicacion4.17
 - > Aplicacion4.18
 - > Aplicacion4.19
 - > Aplicacion5.11
 - > Aplicacion5.12
 - > Aplicacion5.13
 - > Aplicacion5.14
 - > Aplicacion5.15
 - > Aplicacion5.18
 - > Aplicacion6.11
 - > Aplicacion6.12
 - > Aplicacion6.13
 - > Aplicacion6.16
 - > Aplicacion6.17
 - > Aplicacion6.18
 - > Aplicacion6.19
 - > Aplicacion6.20
 - > Apropuestas2.2
 - > Apropuestas2.3
 - > Apropuestas2.4
 - > Apropuestas2.5

...va Aplicacion612.java x Aplicacion613.java x Aplicacion616.java x Aplicacion617.java x Aplicacion618.java x Aplicacion619.java x Aplicacion620.java x

Source History

```
1  ...4 lines
5  package aplicacion6.pkg20;
6
7  import java.util.Arrays;
8  import java.util.Scanner;
9
10 ...4 lines */
14 public class Aplicacion620 {
15
16     /**
17      * @param args the command line arguments
18      */
19     /**
20      * Implementa un programa que lea una frase y muestre todas sus palabras ordenadas de
21      * forma alfabética. Suponemos que cada palabra de la frase se separa de otra por un
22      * único espacio.
23      */
24     public static void main(String[] args) {
25         //pedimos se introduzca la frase
26         Scanner sc = new Scanner(System.in);
27         System.out.println("Introduzca una frase: ");
28         String frase = sc.nextLine();
29         System.out.println();
30         //Guardamos las palabras de la frase en un array que se separan por un espacio en blanco
31         String[] palabras = frase.split(" ");
32         //ordenamos las palabras del array
33         Arrays.sort(palabras, String.CASE_INSENSITIVE_ORDER);
34         //Imprimir la frase ordenada alfabeticamente
35         System.out.println("Frase ordenada alfabeticamente: ");
36         for (String palabra : palabras){
37             System.out.print(palabra + " ");
38         }
39         System.out.println();
40     }
41 }
42
43
```

Output - Aplicacion6.20 (run) x

```
run:
Introduzca una frase:
El desarrollo web abarca la creación de sitios y aplicaciones que permiten a los usuarios interactuar con contenido y servicios digitales

Frase ordenada alfabeticamente:
a abarca aplicaciones con contenido creación de desarrollo digitales El interactuar la los permiten que servicios sitios usuarios web y y
BUILD SUCCESSFUL (total time: 1 minute 3 seconds)
```